

Desarrollo_Web

T.S. Christopher Jara

2026-06-09

Table of contents

Prefacio	4
1 Introducción	5
2 Conociendo Visual Studio Code	7
2.1 ¿Qué es Visual Studio Code?	7
2.2 ¿Para qué sirve Visual Studio Code?	7
2.3 Importancia de VS Code en la programación actual	8
2.4 Manual de Instalación de Visual Studio Code	8
2.4.1 1. Requisitos previos	8
2.4.2 2. Descargar Visual Studio Code	8
2.4.3 3. Instalación en Windows	9
2.4.4 4. Instalación en macOS	9
2.4.5 5. Instalación en Linux	9
2.4.6 6. Verificación de instalación	10
2.5 ¿Cómo configurar Visual Studio Code para Desarrolladores Frontend?	10
2.6 Configuración inicial de Visual Studio Code	10
2.7 Extensiones recomendadas para desarrolladores frontend	11
2.7.1 1. Color Highlight	11
2.7.2 2. GitLens — Git supercharged	11
2.7.3 3. indent-rainbow	11
2.7.4 4. Live Preview	11
2.7.5 5. Live Server	11
2.7.6 6. Material Icon Theme	12
2.7.7 7. Monokai Dark Vibrant	12
2.7.8 8. Prettier - Code formatter	12
2.8 Configuración adicional recomendada	12
3 Guía Maestra de Git: Un Resumen Exhaustivo desde los Fundamentos hasta las Técnicas Avanzadas	13
3.1 Sección 1: Principios Fundamentales del Control de Versiones con Git	13
3.1.1 1.1 El Propósito de un Sistema de Control de Versiones (SCV)	13
3.1.2 1.2 La Arquitectura Distribuida y Filosofía de Git	14
3.1.3 1.3 Configuración Inicial Esencial para el Desarrollador (<code>git config</code>)	14

3.2	Sección 2: El Flujo de Trabajo Básico en Git	15
3.2.1	2.1 Creación y Adquisición de Repositorios	15
3.2.2	2.2 El Ciclo de Vida de los Cambios: Los Tres Estados	15
3.2.3	2.3 Registro de Cambios: El Flujo <code>add</code> , <code>status</code> , <code>commit</code>	16
3.2.4	2.4 Gestión de Archivos y Exclusiones	16
3.3	Sección 3: Navegación y Gestión del Historial del Proyecto	17
3.3.1	3.1 Visualización del Historial de Commits con <code>git log</code>	17
3.3.2	3.2 Inspección Detallada de Cambios (<code>git diff</code> , <code>git show</code>)	18
3.3.3	3.3 Técnicas para Deshacer Cambios	18
3.4	Sección 4: El Poder de la Ramificación en Git	19
3.4.1	4.1 Conceptos Clave: Ramas como Punteros Ligeros	19
3.4.2	4.2 Fusión de Ramas (<code>git merge</code>)	20
3.4.3	4.3 Reorganización del Historial con <code>git rebase</code>	20
3.4.4	4.4 Estrategias y Flujos de Trabajo con Ramas	20
3.5	Sección 5: Colaboración a Través de Repositorios Remotos	21
3.5.1	5.1 Administración de Conexiones Remotas (<code>git remote</code>)	21
3.5.2	5.2 Sincronización de Trabajo: <code>fetch</code> , <code>pull</code> , y <code>push</code>	21
3.5.3	5.3 Trabajo con Ramas Remotas y de Seguimiento	21
3.5.4	5.4 Etiquetado de Versiones y Lanzamientos (<code>git tag</code>)	22
3.6	Sección 6: Herramientas Avanzadas y Técnicas de Maestría	22
3.6.1	6.1 Selección Avanzada de Revisiones	22
3.6.2	6.2 Reescritura del Historial	22
3.6.3	6.3 Depuración de Código con Git	23
3.6.4	6.4 Gestión de Trabajo Temporal con <code>git stash</code>	23
3.7	Sección 7: Personalización, Automatización e Internos de Git	23
3.7.1	7.1 Configuración Avanzada y Atributos de Git (<code>.gitattributes</code>)	23
3.7.2	7.2 Automatización de Flujos de Trabajo con Git Hooks	24
3.7.3	7.3 Una Mirada a los Internos de Git	24
3.8	Sección 8: Git en el Ecosistema de Desarrollo	25
3.8.1	8.1 Gestión de Dependencias con Submódulos	25
3.8.2	8.2 Estrategias de Migración desde Otros Sistemas	25
4	Summary	26
	References	27

Prefacio

El presente libro tiene como objetivo fundamental ofrecer una ruta de aprendizaje completa y estructurada para la formación de los estudiantes de tercero de bachillerato, orientándolos hacia la certificación como desarrolladores frontend. Este manual está diseñado con el propósito de capacitar a los estudiantes de esta etapa académica en las competencias esenciales relacionadas con el desarrollo web, particularmente en tecnologías frontend.

La motivación principal para la elaboración de este libro radica en la carencia de documentación gratuita y accesible que facilite a los jóvenes una formación integral en desarrollo web. Por ello, se ha concebido esta obra como una contribución educativa que busca formar a los futuros programadores del país, proveyéndoles las bases teóricas y prácticas necesarias para su desarrollo profesional.

Este libro está dirigido específicamente a los estudiantes de tercero de bachillerato de la Unidad Educativa “Carlos Crespi II”, con un enfoque pedagógico adaptado a sus necesidades y perfil académico. Se ha diseñado para aquellos lectores que no poseen conocimientos previos en desarrollo web, facilitando su aprendizaje desde los conceptos más elementales hasta técnicas y herramientas actuales.

La estructura del contenido contempla sesiones de trabajo segmentadas en bloques de 45 minutos de aprendizaje guiado con acompañamiento docente, seguidos de 45 minutos de práctica supervisada y culminando con 90 minutos de aprendizaje autónomo. Este enfoque metodológico busca optimizar la asimilación de conocimientos y habilidades, garantizando un aprendizaje progresivo y efectivo al ser leído y ejecutado en secuencia.

Confiamos en que esta obra será un recurso invaluable para la formación académica y técnica de los estudiantes, aportando al desarrollo de competencias que les permitan integrarse exitosamente en el ámbito profesional del desarrollo web frontend.

Christopher Jara
Autor

1 Introducción

En la actualidad, el desarrollo web ocupa un lugar esencial en el mundo laboral, constituyendo una de las áreas con mayor demanda y crecimiento en el sector tecnológico. Las empresas de todos los tamaños y sectores dependen cada vez más de plataformas digitales para conectar con sus clientes, ofrecer servicios y potenciar su competitividad. Esta realidad convierte a los desarrolladores web en profesionales altamente valorados y solicitados en el mercado global.

El propósito de esta obra es dotar a los estudiantes de tercero de bachillerato de la Unidad Educativa “Carlos Crespi II” con una formación sólida, que les permita certificar sus competencias como desarrolladores frontend y así acceder a las oportunidades laborales que ofrece el mundo digital moderno. A través de una ruta de aprendizaje progresiva y práctica, este libro busca transformar a jóvenes sin conocimientos previos en profesionales capaces, motivados y preparados para los retos actuales.

Existen diversos tipos de programadores web, y conocer sus habilidades y roles permitirá a los estudiantes comprender las variadas oportunidades y caminos en esta carrera. Los **desarrolladores frontend** se encargan de la interfaz y experiencia del usuario, diseñando y creando sitios web atractivos y funcionales con habilidades en diseño visual, usabilidad, y comunicación efectiva. Los **desarrolladores backend** trabajan en la lógica, servidores y bases de datos que sustentan las aplicaciones web, demandando habilidades analíticas, pensamiento estructurado y dominio de la gestión de datos. Finalmente, los **desarrolladores full stack** combinan ambos roles, siendo versátiles y adaptativos, con capacidad para integrar y gestionar tanto la parte visible como la funcional del software, lo que requiere una gran disciplina, aprendizaje constante y capacidad de resolución de problemas complejos.

El desarrollo web ha evolucionado considerablemente desde sus inicios, pasando de simples páginas estáticas a aplicaciones web interactivas y dinámicas que impulsan la economía digital. Esta transformación crea nuevas exigencias y oportunidades laborales a nivel mundial, con una tendencia clara hacia la profesionalización, especialización y uso de metodologías ágiles en entornos colaborativos. El futuro del desarrollo web se orienta hacia la inclusión de tecnologías emergentes, la inteligencia artificial y una interacción aún más personalizada y eficiente entre usuario y plataforma.

Este libro servirá como guía para que los estudiantes no solo aprendan los fundamentos teóricos y prácticos, sino que también desarrollen habilidades esenciales como el pensamiento crítico, la creatividad, el trabajo en equipo y la autonomía. Al concluir esta ruta educativa, los lectores estarán capacitados para certificarse y convertirse en desarrolladores frontend competentes, abriendo así la puerta a un mercado laboral dinámico y en constante crecimiento.

Confiamos en que esta obra será una herramienta transformadora en la trayectoria académica y profesional de los estudiantes, contribuyendo a la formación de los futuros profesionales del sector tecnológico del país, con la motivación y preparación necesaria para destacar en el mundo laboral actual y futuro.

2 Conociendo Visual Studio Code

2.1 ¿Qué es Visual Studio Code?

Visual Studio Code (VS Code) es un editor de código fuente gratuito desarrollado por Microsoft, disponible para Windows, macOS y Linux. Se caracteriza por ser liviano, multiplataforma y altamente extensible. VS Code combina funciones avanzadas de un entorno de desarrollo integrado (IDE) con la sencillez de un editor de texto moderno, permitiendo trabajar con múltiples lenguajes de programación y tecnologías en un entorno centralizado[web:29][web:33][web:34].

2.2 ¿Para qué sirve Visual Studio Code?

VS Code se utiliza principalmente para:

- **Escribir código fuente:** Proporciona herramientas como resaltado de sintaxis, auto-completado inteligente y depuración.
 - **Gestionar proyectos de software:** Ofrece integración con Git y otros sistemas de control de versiones.
 - **Extender funcionalidades:** Permite instalar extensiones para soportar frameworks, nuevos lenguajes, temas visuales y herramientas especializadas.
 - **Ejecutar comandos:** Cuenta con terminal integrada y paneles interactivos para configurar y automatizar tareas del proyecto[web:30][web:31][web:35].
-

2.3 Importancia de VS Code en la programación actual

Visual Studio Code es uno de los editores más usados por desarrolladores en todo el mundo debido a su combinación de eficiencia, flexibilidad y una comunidad activa. Sus ventajas incluyen:

- **Multiplataforma:** Se ejecuta en sistemas operativos populares, permitiendo equipos diversos trabajar con la misma herramienta.
 - **Extensibilidad:** Dispone de miles de extensiones y plugins desarrollados por la comunidad y por empresas líderes.
 - **Productividad:** Optimiza la escritura y gestión del código, reduce errores y facilita el trabajo colaborativo.
 - **Adopción profesional y académica:** Es un estándar en la industria, utilizado tanto por grandes compañías como por instituciones educativas para enseñar programación y desarrollo de software[web:29][web:34][web:35].
-

2.4 Manual de Instalación de Visual Studio Code

2.4.1 1. Requisitos previos

- Conexión a internet
 - Sistema operativo compatible: Windows 7 o superior, macOS 10.11 o superior, o distribuciones comunes de Linux
-

2.4.2 2. Descargar Visual Studio Code

1. Abre tu navegador de preferencia.
 2. Accede a la página oficial: <https://code.visualstudio.com/>
 3. Haz clic en “Download” y selecciona el instalador compatible para tu sistema operativo (Windows, macOS, o Linux).
-

2.4.3 3. Instalación en Windows

1. Ejecuta el archivo descargado (`VSCodeUserSetup-x64-*.exe`).
 2. Acepta los términos y condiciones.
 3. Selecciona la carpeta de destino (puedes dejar la predeterminada).
 4. Activa opciones recomendadas como “Agregar al menú contextual” (para abrir carpetas directamente desde el explorador).
 5. Haz clic en “Instalar” y espera a que finalice.
 6. Ejecuta VS Code seleccionando “Finalizar” y marcando “Abrir Visual Studio Code”.
-

2.4.4 4. Instalación en macOS

1. Descomprime el archivo `.zip` descargado.
 2. Mueve el icono de Visual Studio Code a la carpeta de “Aplicaciones”.
 3. Abre la aplicación desde “Launchpad” o “Aplicaciones”.
 4. (Opcional) Abre la paleta de comandos (`Cmd + Shift + P`) y escribe:
`Shell Command: Install 'code' command in PATH`
para poder ejecutar VS Code desde la Terminal con el comando `code`.
-

2.4.5 5. Instalación en Linux

2.4.5.1 Ubuntu / Debian

1. Descarga e instala el paquete `.deb` desde la web oficial.
2. Alternativamente, desde Terminal: `sudo apt update` `sudo apt install code`

Si instalas desde el paquete descargado: `sudo dpkg -i nombre_paquete.deb`

Reemplaza `nombre_paquete.deb` con el archivo descargado.

2.4.5.2 Fedora / Red Hat / CentOS

1. Descarga e instala el paquete `.rpm` desde la web oficial.
2. Instala ejecutando:

`sudo dnf install code`

2.4.6 6. Verificación de instalación

- Abre Visual Studio Code.
- Comprueba que la ventana principal aparece correctamente.
- Verifica la versión ejecutando en la terminal integrada:

```
code --version
```

2.5 ¿Cómo configurar Visual Studio Code para Desarrolladores Frontend?

Visual Studio Code (VS Code) es un editor de código muy popular entre desarrolladores frontend por su ligereza, flexibilidad y capacidad de personalización mediante extensiones. Configurar correctamente VS Code aumenta la productividad y mejora la experiencia de codificación.

A partir de este punto le voy a guiar sobre la configuración básica y recomendaciones de extensiones esenciales para desarrolladores frontend.

2.6 Configuración inicial de Visual Studio Code

1. Instalación de VS Code

Descarga e instala Visual Studio Code desde su sitio oficial: <https://code.visualstudio.com/>

2. Abrir VS Code y acceder a extensiones

- En la barra lateral izquierda, selecciona el icono de extensiones () o presiona `Ctrl+Shift+X`.
- Utiliza la barra de búsqueda para encontrar extensiones.

3. Temas y personalización

- Cambia el tema desde `Archivo > Preferencias > Tema de Color` o con `Ctrl+K Ctrl+T`.
 - Se recomienda elegir temas legibles y que reduzcan la fatiga visual para largas sesiones.
-

2.7 Extensiones recomendadas para desarrolladores frontend

2.7.1 1. Color Highlight

- **Descripción:** Resalta los colores definidos en código CSS, SCSS, Less, y archivos relacionados, mostrando el color de manera visual junto al valor hexadecimal o RGB.
- **Uso:** Facilita la identificación y selección de colores en hojas de estilos y archivos de diseño.
- **Instalación:** Buscar Color Highlight en la tienda de extensiones e instalar.

2.7.2 2. GitLens — Git supercharged

- **Descripción:** Amplía las capacidades de Git integradas en VS Code, mostrando quién cambió qué línea y cuándo, historial de archivos, comparaciones de ramas, y más.
- **Uso:** Perfecto para seguimiento, revisión de código y trabajo colaborativo en proyectos con control de versiones.
- **Instalación:** Buscar GitLens e instalar.

2.7.3 3. indent-rainbow

- **Descripción:** Colorea las sangrías (indentaciones) en el código, usando diferentes colores para cada nivel.
- **Uso:** Ayuda a visualizar indentaciones complejas y a mejorar la legibilidad del código anidado (especialmente en HTML, CSS, JavaScript).
- **Instalación:** Buscar indent-rainbow.

2.7.4 4. Live Preview

- **Descripción:** Permite visualizar en tiempo real el resultado de los archivos HTML directamente en VS Code.
- **Uso:** Muy útil para verificar rápidamente los cambios sin necesidad de cambiar de aplicación o navegador.
- **Instalación:** Buscar Live Preview.

2.7.5 5. Live Server

- **Descripción:** Lanza un servidor local que refresca automáticamente el navegador al guardar archivos.
- **Uso:** Ideal para desarrollo rápido y pruebas en proyectos web locales, con recarga instantánea.

- **Instalación:** Buscar `Live Server`.

2.7.6 6. Material Icon Theme

- **Descripción:** Cambia los iconos predeterminados con un conjunto moderno y visualmente agradable inspirado en Material Design.
- **Uso:** Facilita la identificación rápida de tipos de archivos y carpetas en el explorador de VS Code.
- **Instalación:** Buscar `Material Icon Theme`.

2.7.7 7. Monokai Dark Vibrant

- **Descripción:** Tema de color para VS Code basado en Monokai, con colores vibrantes y alto contraste para reducir fatiga visual.
- **Uso:** Recomendado para sesiones largas de programación, especialmente en ambientes con poca luz.
- **Instalación:** Buscar `Monokai Dark Vibrant`.

2.7.8 8. Prettier - Code formatter

- **Descripción:** Formateador automático de código que aplica estilos consistentes siguiendo reglas definidas o personalizadas.
 - **Uso:** Facilita mantener un código limpio y uniforme en proyectos colaborativos.
 - **Instalación:** Buscar `Prettier - Code formatter`.
 - **Configuración:** Se recomienda habilitar el formato automático al guardar.
-

2.8 Configuración adicional recomendada

- **Configura el linting y validación en JavaScript/TypeScript** para detectar errores temprano (por ejemplo con ESLint).
 - **Atajos de teclado personalizados** para acelerar tareas frecuentes.
 - **Sincronización de configuraciones** para mantener la misma configuración en diferentes dispositivos (usando cuenta Microsoft o GitHub).
-

3 Guía Maestra de Git: Un Resumen Exhaustivo desde los Fundamentos hasta las Técnicas Avanzadas

3.1 Sección 1: Principios Fundamentales del Control de Versiones con Git

3.1.1 1.1 El Propósito de un Sistema de Control de Versiones (SCV)

Un **Sistema de Control de Versiones (SCV)**, o Version Control System (VCS), es una aplicación esencial que **gestiona y registra los cambios** realizados sobre los archivos de un proyecto a lo largo del tiempo. Su objetivo es: * Recuperar **versiones específicas** o revertir el proyecto a un estado anterior. * Comparar modificaciones e identificar **quién introdujo un cambio y cuándo**. * Facilitar la **colaboración** entre múltiples desarrolladores.

3.1.1.1 Modelos de SCV

Modelo	Descripción	Ejemplos	Punto Crítico
Locales	Operan en una única máquina, manteniendo parches locales.	RCS	No aptos para colaboración.
Centralizados (CVCS)	Utilizan un servidor central con todo el historial. Los clientes “hacen checkout”.	Subversion (SVN), Perforce	Punto único de fallo. Si el servidor cae o se corrompe, la colaboración se detiene y se corre el riesgo de pérdida total del historial.

Modelo	Descripción	Ejemplos	Punto Crítico
Distribuidos (DVCS)	Cada cliente clona un espejo completo del repositorio, incluyendo el historial íntegro.	Git , Mercurial	Mayor robustez (cada clon es una copia de seguridad) y permite trabajo desconectado y flujos más flexibles.

3.1.2 1.2 La Arquitectura Distribuida y Filosofía de Git

El diseño de Git se basa en la **velocidad**, la **integridad** y un potente sistema de **ramificación**.

- **Modelo de Instantáneas (Snapshots), no Diferencias:** Git almacena la información como una serie de **instantáneas** del sistema de archivos. Cada **commit** captura una imagen del estado de todos los archivos. Si un archivo no ha cambiado, Git solo almacena un enlace a la versión anterior idéntica para ser eficiente.
- **Operaciones Locales:** La mayoría de las operaciones (**log**, **diff**, **branch**) se realizan localmente sin red, gracias a que cada clon tiene el historial completo, lo que las hace **casi instantáneas**.
- **Integridad de Datos:** Cada archivo y **commit** se verifica con un **hash SHA-1**. Es criptográficamente imposible alterar el contenido sin que Git lo detecte.
- **Modelo Aditivo:** Las acciones en Git solo **añaden datos** a la base de datos, lo que hace que sea difícil perder trabajo permanentemente y fomenta la experimentación.

3.1.3 1.3 Configuración Inicial Esencial para el Desarrollador (`git config`)

El comando `git config` gestiona ajustes en tres niveles jerárquicos (**Sistema** → **Global** → **Local**):

Configuración	Comando de Ejemplo	Propósito
Identidad del Usuario	<code>git config --global user.name "John Doe" git config --global user.email johndoe@example.com</code>	Crucial: Esta información se incrusta de forma inmutable en cada commit.
Editor Predeterminado	<code>git config --global core.editor emacs</code>	Define el editor para mensajes de commit.
Rama por Defecto	<code>git config --global init.defaultBranch main</code>	Cambia el nombre de la rama inicial (p. ej., de master a main).
Verificación	<code>git config --list --show-origin</code>	Revisa la configuración actual y su origen.

3.2 Sección 2: El Flujo de Trabajo Básico en Git

3.2.1 2.1 Creación y Adquisición de Repositorios

1. Inicialización de un Repositorio Nuevo:

- `git init`: Crea el subdirectorio oculto `.git` (la base de datos y metadatos) en un directorio existente.

2. Clonación de un Repositorio Existente:

- `git clone <url>`: Descarga una **copia local completa** del repositorio, incluyendo todo el historial.

3.2.2 2.2 El Ciclo de Vida de los Cambios: Los Tres Estados

El flujo de trabajo de Git se define por tres áreas:

1. **Directorio de Trabajo (Working Tree)**: La versión específica del proyecto en la que se está **editando**.
2. **Área de Staging (Staging Area o Index)**: Una zona intermedia que almacena los cambios **que se incluirán en el próximo commit**. Permite seleccionar y agrupar cambios de forma precisa.

3. **Directorio Git (Repositorio):** El directorio `.git` donde se almacena permanentemente la base de datos de objetos (el historial).

Estado del Archivo	Descripción
untracked	Archivo nuevo que Git no está rastreando.
unmodified	Archivo rastreado, sin cambios desde el último commit.
modified	Archivo rastreado, con cambios en el directorio de trabajo.
staged	Archivo modificado o nuevo, preparado para ser confirmado.

3.2.3 2.3 Registro de Cambios: El Flujo `add`, `status`, `commit`

El ciclo fundamental es:

1. **Inspeccionar:**

- `git status`: Muestra el estado de los archivos (modificados, staged, no rastreados). La opción `-s` (`--short`) proporciona una vista compacta.

2. **Preparar (Staging):**

- `git add <archivo>`: Inicia el seguimiento de archivos o **añade el contenido modificado** al Área de Staging. La clave es que el staging permite construir **commits atómicos** y coherentes, agrupando solo los cambios relacionados.

3. **Confirmar:**

- `git commit`: Toma la instantánea del Área de Staging y la guarda **permanentemente** en el repositorio.
 - `git commit -m "Mensaje"`: Proporciona el mensaje directamente.
 - `git commit -a`: Combina `add` de todos los archivos **ya rastreados** y `commit` en un solo paso.
-

3.2.4 2.4 Gestión de Archivos y Exclusiones

Comando/Archivo	Propósito	Ejemplo
<code>.gitignore</code>	Evita que Git rastree archivos generados (logs, compilaciones). Contiene patrones.	<code>*.log, /temp/</code>
<code>git rm</code>	Elimina el archivo del directorio de trabajo y lo prepara para ser eliminado en el próximo commit.	<code>git rm mi_archivo.txt</code>
<code>git rm --cached</code>	Mantiene el archivo en el directorio de trabajo, pero lo elimina del control de versiones.	<code>git rm --cached binario</code>
<code>git mv</code>	Convención que ejecuta <code>rm</code> del original y <code>add</code> del nuevo. Git detecta el renombrado.	<code>git mv antiguo.js nuevo.js</code>

3.3 Sección 3: Navegación y Gestión del Historial del Proyecto

3.3.1 3.1 Visualización del Historial de Commits con `git log`

Opción de <code>git log</code>	Descripción
<code>--pretty=oneline</code>	Muestra cada commit en una sola línea (hash y mensaje).
<code>--graph --decorate</code>	Dibuja un grafo ASCII y muestra nombres de ramas y etiquetas.
<code>-p o --patch</code>	Muestra el diff (los cambios) introducidos en cada commit.
<code>--stat</code>	Muestra un resumen estadístico de los archivos modificados.
<code>-n</code>	Limita la salida a los últimos <i>n</i> commits (p. ej., -2).
<code>--since, --author, --grep</code>	Opciones para filtrar por fecha, autor, o texto en el mensaje.

3.3.2 3.2 Inspección Detallada de Cambios (git diff, git show)

- `git diff`: Muestra los cambios en el Directorio de Trabajo **no** añadidos al Staging.
- `git diff --staged` (o `--cached`): Muestra los cambios en el **Staging** listos para el commit.
- `git diff HEAD`: Muestra **todos** los cambios (Staging + Directorio de Trabajo) desde el último commit.
- `git show <commit>`: Muestra los metadatos y los **cambios** de un commit específico.

3.3.3 3.3 Técnicas para Deshacer Cambios

Comando	Alcance	Efecto en Directorio de Trabajo	Efecto en Staging Area (Index)	Efecto en Puntero de Rama (HEAD)	Caso de Uso Típico
<code>git reset HEAD <file></code>	Archivo	Sin cambios	Actualiza (deshace add)	Sin cambios	Deshacer un <code>git add</code> (<code>unstage</code>).
<code>git checkout -- <file></code>	Archivo	Sobrescribe con la versión del Index/último commit.	Sin cambios	Sin cambios	Descartar cambios locales no deseados en un archivo.
<code>git commit --amend</code>	Proyecto	Sin cambios	Actualiza con nuevos add	Reemplaza el último commit (reescribe la historia).	Corregir rápidamente el último commit (mensaje o contenido).
<code>git reset --hard <commit></code>	Proyecto	Sobrescribe para coincidir con <code><commit></code> .	Actualiza para coincidir con <code><commit></code> .	Mueve HEAD a <code><commit></code> .	Descartar completamente commits y todos los cambios locales. Peligroso.

Comando	Alcance	Efecto en Directorio de Trabajo	Efecto en Staging Area (Index)	Efecto en Rama (HEAD)	Caso de Uso Típico
<code>git reset --mixed <commit></code>	Proyecto	Sin cambios	Actualiza para coincidir con <commit>.	Mueve HEAD a <commit>.	Deshacer commits y mantener los cambios en el directorio de trabajo (vuelven a modified).
<code>git revert <commit></code>	Proyecto	Actualiza	Actualiza	Crea un nuevo commit y avanza HEAD.	Deshacer un commit en un historial público de forma segura (no reescribe la historia).

3.4 Sección 4: El Poder de la Ramificación en Git

3.4.1 4.1 Conceptos Clave: Ramas como Punteros Ligeros

Una rama es un **puntero móvil y ligero** que apunta a un commit. Crear una rama es una operación casi instantánea. El puntero especial **HEAD** indica la rama local actual.

Comando de Rama	Propósito
<code>git branch <nombre></code>	Crea una nueva rama.
<code>git branch</code>	Lista las ramas y muestra la activa (*).
<code>git checkout <nombre></code>	Cambia a la rama (actualiza el directorio de trabajo).
<code>git checkout -b <nombre></code>	Atajo: Crea una nueva rama y cambia a ella.
<code>git branch -d <nombre></code>	Elimina una rama fusionada. (-D para forzar la eliminación).

3.4.2 4.2 Fusión de Ramas (`git merge`)

El proceso para integrar cambios de una rama a otra.

- **Fusión Fast-Forward:** Ocurre si la rama a fusionar está directamente por delante. Git solo mueve el puntero de la rama actual hacia adelante.
 - **Fusión de Tres Vías (Three-Way Merge):** Las historias han divergido. Git utiliza la punta de cada rama y su ancestro común. El resultado es un **Merge Commit** con dos padres.
 - **Gestión de Conflictos:** Si dos ramas modifican la misma parte del mismo archivo, Git no puede fusionarlas. El desarrollador debe:
 1. Editar el archivo con los marcadores (<<<<<<<, =====, >>>>>>).
 2. `git add <archivo>` para marcar el conflicto como resuelto.
 3. `git commit` para completar la fusión.
-

3.4.3 4.3 Reorganización del Historial con `git rebase`

`git rebase` toma los cambios de una rama y los **reproduce secuencialmente** sobre otra, resultando en un historial **lineal y limpio** al reescribir la historia.

- **Merge vs. Rebase:**
 - **Merge:** Preserva la historia real con todas sus bifurcaciones (historial más complejo pero preciso).
 - **Rebase:** Crea un historial más simple y legible (reescribe la historia).
 - **La Regla de Oro del Rebasado: Nunca** se deben rebasar commits que ya han sido **empujados a un repositorio público compartido**. Esto lleva a historiales divergentes para los colaboradores.
-

3.4.4 4.4 Estrategias y Flujos de Trabajo con Ramas

- **Ramas de Larga Duración:** Se mantienen para diferentes etapas (`main` para código estable, `develop` para integración).
 - **Ramas de Tópico (Topic Branches):** Ramas de corta duración para una **única funcionalidad o corrección**. Permiten el desarrollo aislado y facilitan la revisión de código.
-

3.5 Sección 5: Colaboración a Través de Repositorios Remotos

3.5.1 5.1 Administración de Conexiones Remotas (`git remote`)

El comando gestiona las conexiones. Por convención, el remoto clonado se llama **origin**.

- `git remote -v`: Lista las conexiones remotas existentes.
- `git remote add <nombre> <url>`: Añade un nuevo remoto.

3.5.2 5.2 Sincronización de Trabajo: `fetch`, `pull`, y `push`

Comando	Acción	Descripción
<code>git fetch</code>	Descargar, NO fusionar	Baja objetos y referencias del remoto al repositorio local, actualizando las Ramas de Seguimiento Remoto (<code>origin/master</code>). Operación segura.
<code>git pull</code>	Descargar Y fusionar	Conveniencia que ejecuta <code>git fetch</code> y luego <code>git merge</code> (o <code>rebase</code>).
<code>git push <remoto> <rama></code>	Subir commits	Envía los commits locales de una rama al repositorio remoto. Puede ser rechazado si no es un fast-forward (hay cambios remotos).

3.5.3 5.3 Trabajo con Ramas Remotas y de Seguimiento

- **Ramas de Seguimiento Remoto**: Referencias locales de **solo lectura** del estado de las ramas en el remoto (p. ej., `origin/main`).
- **Ramas de Seguimiento (Tracking Branches)**: Ramas locales con una relación directa con una remota. Permiten `git pull` y `git push` sin argumentos.
- Eliminar rama remota: `git push origin --delete <rama>`.

3.5.4 5.4 Etiquetado de Versiones y Lanzamientos (`git tag`)

Se usan para marcar puntos estables en el historial (p. ej., `v1.0`).

- **Etiquetas Anotadas:** Objetos completos con autor, fecha, mensaje. Se crean con `git tag -a`.
 - Para compartirlas, deben ser empujadas explícitamente: `git push --tags`.
-

3.6 Sección 6: Herramientas Avanzadas y Técnicas de Maestría

3.6.1 6.1 Selección Avanzada de Revisiones

- **Referencias de Ascendencia:**
 - `HEAD^`: El padre de `HEAD`.
 - `HEAD~3`: El “bisabuelo” (tres pasos hacia atrás del primer padre).
 - **Rangos de Commits:**
 - `A..B`: Commits alcanzables desde *B* pero **no** desde *A* (útil: `master..feature`).
 - `A...B`: Commits que han divergido de *A* o *B*, pero no de ambos.
 - **RefLog:** Registro local de dónde han estado `HEAD` y los punteros. Permite recuperar trabajo perdido: `HEAD@{5}` (quinto estado anterior).
-

3.6.2 6.2 Reescritura del Historial

- **Rebasado Interactivo (`git rebase -i`):** Herramienta potente para editar una serie de commits locales antes de compartirlos. Permite **reordenar**, **eliminar (drop)**, **combinar (squash o fixup)** y **editar** commits.
 - **`git filter-branch`:** Comando de scripting para reescribir grandes porciones del historial (p. ej., eliminar un archivo de cada commit). **Destructivo**.
-

3.6.3 6.3 Depuración de Código con Git

- **git blame <archivo>:** Anota cada línea para mostrar qué commit y qué autor la modificaron por última vez.
 - **git bisect:** Automatiza una **búsqueda binaria** en el historial para encontrar el **commit exacto** que introdujo un error.
 1. **git bisect bad** (commit con error)
 2. **git bisect good** (commit sin error)
 3. Git guía la inspección hasta el culpable.
 - **git grep:** Búsqueda rápida de texto en todos los archivos rastreados, incluyendo versiones antiguas.
-

3.6.4 6.4 Gestión de Trabajo Temporal con git stash

Guarda temporalmente los cambios no confirmados (staged y no-staged) para obtener un Directorio de Trabajo limpio y poder cambiar de contexto rápidamente.

- **git stash:** Guarda los cambios.
 - **git stash list:** Muestra la lista de “almacenes”.
 - **git stash apply** o **git stash pop:** Reaplica o reaplica y elimina el stash.
-

3.7 Sección 7: Personalización, Automatización e Internos de Git

3.7.1 7.1 Configuración Avanzada y Atributos de Git (.gitattributes)

El archivo `.gitattributes` permite definir el comportamiento de Git por ruta:

- **Gestión de Finales de Línea:** Controla la normalización (`crLf` en Windows, `lf` en Linux).
 - **Controladores de diff:** Permite usar herramientas externas para mostrar diferencias en archivos binarios.
-

3.7.2 7.2 Automatización de Flujos de Trabajo con Git Hooks

Los **hooks** son scripts que Git ejecuta automáticamente en puntos clave de su ciclo de vida.

- **Hooks del Lado del Cliente (Máquina Local):**
 - **pre-commit:** Antes de crear el commit (ideal para linters y pruebas).
 - **commit-msg:** Valida el formato del mensaje.
 - **Hooks del Lado del Servidor (Remoto):**
 - **pre-receive / update:** Antes de aceptar un push (para aplicar políticas de código o permisos).
 - **post-receive:** Después de un push exitoso (para despliegues o notificaciones).
-

3.7.3 7.3 Una Mirada a los Internos de Git

La arquitectura de Git se divide en:

- **Porcelana (Porcelain):** Comandos de alto nivel para el día a día (**commit**, **merge**).
- **Plomería (Plumbing):** Comandos de bajo nivel que hacen el trabajo real (para flujos de trabajo personalizados).

3.7.3.1 Objetos de Git (La Base de Datos)

1. **Blob:** Almacena el **contenido** de un archivo.
 2. **Árbol (Tree):** Representa un **directorio**, apuntando a **blobs** y otros **trees**.
 3. **Commit:** La **instantánea** completa. Apunta a un **tree** y contiene metadatos (autor, fecha, mensaje) y punteros a sus **padres**.
 4. **Etiqueta (Tag):** Un puntero con nombre a un **commit**.
- **Packfiles:** Archivos comprimidos que almacenan múltiples objetos usando **compresión delta** para ahorrar espacio y mejorar la eficiencia.
-

3.8 Sección 8: Git en el Ecosistema de Desarrollo

3.8.1 8.1 Gestión de Dependencias con Submódulos

Los **submódulos** permiten incrustar un repositorio Git dentro de otro, vinculando el repositorio principal a un **commit específico** del repositorio dependiente. Se gestionan con `git submodule`.

3.8.2 8.2 Estrategias de Migración desde Otros Sistemas

- **Subversion (SVN):** `git svn` actúa como un cliente de SVN para clonar el historial.
- **Mercurial (Hg):** Herramientas como `hg-fast-export` aprovechan el modelo distribuido similar.
- **git fast-import:** Un comando de plomería para escribir datos directamente en la base de datos de Git, permitiendo crear **scripts de migración personalizados** desde casi cualquier sistema de origen.

4 Summary

In summary, this book has no content whatsoever.

References